Java Basics

Beginner-friendly notes | DSA in Java Series

1. What is Java?

- Java is a **high-level, object-oriented programming language**.
- It was created by **James Gosling** at Sun Microsystems and released in **1995**. It is now owned by Oracle.
- Java is used to build mobile apps (Android), web applications, desktop software, games and enterprise systems.

Key features

- **Simple:** easy syntax compared to C and C++.
- **Object-oriented:** programs are built using classes and objects.
- **Platform independent:** write once, run anywhere (WORA).
- **Secure:** no direct memory access with pointers.
- **Robust:** strong error checking and automatic memory management.
- **Portable and fast:** compiled to bytecode that runs on any JVM.

2. JDK, JRE and JVM

Term	Full form	What it does
JVM	Java Virtual Machine	Runs the compiled Java bytecode on your computer
JRE	Java Runtime Environment	JVM plus the libraries needed to **run** Java programs
JDK	Java Development Kit	JRE plus tools (like the compiler) needed to **write and run** Java programs

Easy way to remember: **JDK contains JRE, and JRE contains JVM.** As a developer, you install the JDK.

3. How a Java Program Runs

- **Step 1:** You write the code in a file ending with **.java** (source code).
- **Step 2:** The **javac** compiler converts it into **bytecode**, stored in a **.class** file.
- **Step 3:** The **JVM** reads the bytecode and runs it on your machine.

```
Hello.java  --javac-->  Hello.class  --java-->  Output
(source code)           (bytecode)              (JVM runs it)
```

Because the bytecode runs on the JVM and not directly on the operating system, the same program works on Windows, Linux and macOS. This is why Java is platform independent.

4. Structure of a Java Program

```
public class Hello {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Output: Hello, World!

Line-by-line meaning

Part	Meaning
public class Hello	Defines a class named Hello. Every Java program lives inside a class.
public static void main(String[] args)	The **main method**. Program execution always starts here.
public	Accessible from anywhere, so the JVM can call it.
static	Can run without creating an object of the class.
void	The method does not return any value.
String[] args	Holds command-line arguments (can be ignored for now).
System.out.println()	Prints text on the screen and moves to a new line.

Rules: the file name must match the public class name (Hello.java). Every statement ends with a **semicolon ;** and Java is **case-sensitive** (Main and main are different).

5. Comments

Comments are notes for humans. The compiler ignores them.

```
// This is a single-line comment

/* This is a
   multi-line comment */
```

6. Variables

- A variable is a **named container that stores data** in memory.
- Its value can change while the program runs.
- In Java you must declare the **data type** before using a variable.

Syntax

```
dataType variableName = value;

int age = 20;
double price = 99.5;
char grade = 'A';
String name = "Piyush";
boolean isPassed = true;
```

Declaration, initialization and assignment

```
int x;        // declaration
x = 10;       // assignment
int y = 5;    // declaration + initialization
x = 25;       // value changed
```

Rules for naming variables

- Can contain letters, digits, underscore _ and dollar **$**.
- **Cannot start with a digit** (1name is wrong, name1 is correct).
- **No spaces** and no special characters like @ or #.
- Cannot be a Java **keyword** (like int, class, for).
- Names are case-sensitive: age and Age are different variables.
- Convention: use **camelCase** (studentName, totalMarks).

7. Data Types

A data type tells Java what kind of value a variable can store. Java has two groups: **primitive** and **non-primitive**.

Primitive data types (8)

Type	Size	Stores	Example
byte	1 byte	Very small whole numbers (-128 to 127)	byte b = 100;
short	2 bytes	Small whole numbers	short s = 5000;
int	4 bytes	Whole numbers (about -2.1 billion to 2.1 billion)	int a = 25;
long	8 bytes	Very large whole numbers (add L)	long l = 5000000000L;
float	4 bytes	Decimal numbers (add f)	float f = 3.14f;
double	8 bytes	Decimal numbers (more precise)	double d = 3.14159;
char	2 bytes	A single character, in single quotes	char c = 'A';
boolean	1 bit*	true or false	boolean ok = true;

*The exact size of boolean depends on the JVM. Think of it as a simple true / false flag.

Non-primitive data types

- **String:** a sequence of characters, written in double quotes. Example: "Hello".
- **Arrays:** store multiple values of the same type.
- **Classes and objects:** user-defined types.

Most used: use **int** for whole numbers, **double** for decimals, **char** for one character, **String** for text and **boolean** for true / false.

Single quotes vs double quotes

```
char c = 'A';        // single quotes: one character
String s = "Apple";  // double quotes: text
```

8. Type Casting

Type casting means converting one data type into another.

Implicit casting (automatic, small to big)

```
byte -> short -> int -> long -> float -> double

int x = 10;
double d = x;       // 10.0  (done automatically)
```

Explicit casting (manual, big to small)

```
double pi = 3.99;
int n = (int) pi;   // 3  (decimal part is cut off, not rounded)
```

Casting a bigger type into a smaller type can **lose data**, so Java needs you to write the cast yourself.

9. Operators

Operators perform operations on values (called operands).

Arithmetic operators

Operator	Name	Example	Result
+	Addition	10 + 3	13
-	Subtraction	10 - 3	7
*	Multiplication	10 * 3	30
/	Division	10 / 3	3 (integer division)
%	Modulus (remainder)	10 % 3	1

When both numbers are **int**, the result of **/** is also an int. 7 / 2 gives 3, but 7.0 / 2 gives 3.5.

Relational operators (result is true or false)

Operator	Meaning	Example
==	Equal to	5 == 5 is true
!=	Not equal to	5 != 3 is true
> <	Greater than, less than	4 > 9 is false
>= <=	Greater or equal, less or equal	7 >= 7 is true

Logical operators

Operator	Name	Meaning
&&	AND	True only if both conditions are true
||	OR	True if at least one condition is true
!	NOT	Reverses true to false and false to true

Assignment operators

Operator	Example	Same as
=	a = 5	Assign 5 to a
+=	a += 3	a = a + 3
-=	a -= 3	a = a - 3
*=	a *= 3	a = a * 3
/=	a /= 3	a = a / 3
%=	a %= 3	a = a % 3

Increment and decrement operators

- **++** increases the value by 1 and **--** decreases it by 1.
- **Post** (a++): use the value first, then change it.
- **Pre** (++a): change the value first, then use it.

```
int x = 5;
int y = x++;    // y = 5, then x becomes 6

int a = 5;
int b = ++a;    // a becomes 6, then b = 6
```

10. Output in Java

Statement	What it does
System.out.println()	Prints and moves to the next line
System.out.print()	Prints and stays on the same line
System.out.printf()	Prints formatted output using format specifiers

```
System.out.println("Hello");
System.out.print("Java ");
System.out.print("is fun");
System.out.println();

int age = 20;
System.out.println("Age is " + age);   // + joins text and value
System.out.printf("Age: %d%n", age);
```

Output:

```
Hello
Java is fun

Age is 20
Age: 20
```

Format specifiers and escape sequences

Symbol	Meaning	Symbol	Meaning
%d	int	\n	New line
%f	float / double	\t	Tab space
%c	char	\"	Double quote
%s	String	\\	Backslash
%n	New line (in printf)	%.2f	Decimal with 2 digits

11. Taking Input using Scanner

- **Scanner** is a class used to read input from the keyboard.
- Import it at the top: **import java.util.Scanner;**
- Create an object: **Scanner sc = new Scanner(System.in);**
- Then call a method to read the type of value you need.

Method	Reads
nextInt()	An int
nextLong()	A long
nextFloat()	A float
nextDouble()	A double
nextBoolean()	A boolean
next()	A single word (stops at space)
nextLine()	A full line of text, including spaces
next().charAt(0)	A single character

Example: read two numbers and print the sum

```
import java.util.Scanner;

public class SumDemo {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter first number: ");
        int a = sc.nextInt();

        System.out.print("Enter second number: ");
        int b = sc.nextInt();

        System.out.println("Sum = " + (a + b));
        sc.close();
    }
}
```

Sample run: Enter first number: 10, Enter second number: 20, Sum = 30

Common problem: if you use **nextLine()** right after **nextInt()**, it reads the leftover new line and skips your input. Fix it by adding an extra **sc.nextLine();** before reading the text.

12. Keywords and Identifiers

- **Keywords** are reserved words with a special meaning in Java, such as **int, class, public, static, void, if, else, for, while, return**. You cannot use them as variable names.
- **Identifiers** are names you give to variables, methods and classes (for example age, totalMarks, Hello).

Naming conventions

Used for	Style	Example
Class	PascalCase	StudentDetails
Variable / method	camelCase	totalMarks, findSum()
Constant	UPPER_CASE	MAX_VALUE

13. Common Mistakes to Avoid

- Forgetting the **semicolon ;** at the end of a statement.
- Writing **system.out.println** instead of **System.out.println** (capital S).
- File name not matching the public class name.
- Using **=** instead of **==** when comparing values.
- Using single quotes for text: 'Hello' is wrong, "Hello" is correct.
- Using a variable before declaring or initializing it.
- Forgetting that **int / int** gives an int (7 / 2 is 3, not 3.5).

14. Basic Practice Programs

Try writing these yourself:

- Print Hello World
- Print your name, age and city on separate lines
- Add, subtract, multiply and divide two numbers
- Swap two numbers (with and without a third variable)
- Find the area and perimeter of a rectangle
- Find the area and circumference of a circle
- Calculate simple interest
- Convert Celsius to Fahrenheit
- Convert kilometers to meters
- Find the average of three numbers
- Print the ASCII value of a character

15. Quick Summary

- Java is object-oriented and platform independent (write once, run anywhere).
- **JDK** contains **JRE**, and **JRE** contains **JVM**.
- Code goes in a **.java** file, **javac** compiles it to **.class** bytecode, and the **JVM** runs it.
- Every program starts from the **main** method.
- Variables need a data type: int, double, char, boolean, String.
- Use casting to convert between types. Big to small needs a manual cast.
- Operators: arithmetic, relational, logical, assignment, increment and decrement.
- Use **print / println / printf** for output and **Scanner** for input.

Strong basics make everything else easier. Type every program yourself and run it.